COMP3028 Lecture 11 — Docker Lab: LocalLoop Node.js API

This **beginner-friendly teaching project** uses the Spring 2026 LocalLoop Community Food Rescue Platform as a context for learning how to package and distribute an application with Docker. It implements a **small subset** of the case study, not the complete production application.

The example deliberately uses **Node.js built-in modules only** (`node:http`, `node:events`, `node:test`) so there are **no external npm packages** to install. This lets students focus on Docker first. The API stores food listings **in memory only**: data resets when a new process/container starts. Authentication, food safety approval, a durable database and production monitoring are not implemented.

1. What you will build

- `GET /` — application information
- `GET /health` — health response
- `GET /api/listings` — list food entries
- `GET /api/stats` — count entries by status
- `POST /api/listings` — create a food entry
- `POST /api/listings/:id/reserve` — reserve an entry; duplicate reservation returns 409
- `listing.created` and `listing.reserved` events (logged locally, not durable)

Files:

```
localloop-api/
  src/app.js              Node HTTP handler and in-memory data
  src/server.js           Entry point and port configuration
  test/app.test.js        Three automated API test cases
  package.json            npm commands
  package-lock.json       Reproducible npm manifest
  Dockerfile              Container build recipe
  .dockerignore           Files excluded from Docker build context
  compose.yaml            Repeatable container configuration
  README.md               This handout
```

2. Prerequisites

Install Docker Desktop (Windows/macOS) or Docker Engine with the Compose plugin (Linux). Start Docker. Install Node.js 22 or later to execute the app locally (the image uses `node:24-alpine`). You also need a code editor, a terminal and internet for the first image pull. Docker Hub distribution is optional.

In a terminal, verify:

```
docker --version
docker info
docker compose version
node --version
npm --version
```

If `docker info` fails with a daemon connection error, start Docker Desktop or Docker Engine before continuing.

3. Run and test Node.js locally

Open a terminal **inside the `localloop-api` directory**:

```
npm ci
npm test
npm start
```

Expected test result: **3 passed, 0 failed**. Expected application message: `LocalLoop API listening on port 3000`.

Keep this terminal running and open these URLs in your browser:

- http://localhost:3000/health
- http://localhost:3000/api/listings
- http://localhost:3000/api/stats

Expected `/health` response:

```
{"status":"ok","service":"localloop-api"}
```

3a. Create a food listing (macOS/Linux/Git Bash)

```
curl -i -X POST http://localhost:3000/api/listings \
  -H "Content-Type: application/json" \
  -d '{"title":"Fruit box","quantity":4,"pickupLocation":"Campus library"}'
```

Expected HTTP status: **201 Created**. Then open `http://localhost:3000/api/listings` and see the new listing.

3b. Create a food listing (Windows PowerShell)

```
$body = @{title="Fruit box"; quantity=4; pickupLocation="Campus library"} | ConvertTo-Json
Invoke-RestMethod -Uri http://localhost:3000/api/listings -Method POST -ContentType "application/json" -Bo
```

3c. Reserve a listing

macOS/Linux/Git Bash:

```
curl -i -X POST http://localhost:3000/api/listings/1/reserve
curl -i -X POST http://localhost:3000/api/listings/1/reserve
```

Windows PowerShell:

```
Invoke-WebRequest -Uri http://localhost:3000/api/listings/1/reserve -Method POST
```

First request: **200 OK**. Second request: **409 Conflict**. In PowerShell, the second request may throw an HTTP error; that is the expected conflict response.

Stop the local Node server using Ctrl+C before the next section so port 3000 is available.

4. Build a Docker image

```
docker build -t localloop-api:1.0 .
docker images
```

Meaning: `docker build` reads `Dockerfile`; `-t` names the resulting image; `.` means use the current directory as the build context. The Dockerfile starts from Node 24 Alpine, prepares the npm manifest, copies the server, switches to non-root `node`, and launches `src/server.js`.

Checkpoint: `docker images` includes `localloop-api:1.0`.

5. Run, inspect and test the container

```
docker run -d --name localloop -p 3000:3000 localloop-api:1.0
docker ps
docker logs localloop
docker exec localloop node --version
```

Visit `http://localhost:3000/health` and `http://localhost:3000/api/listings`.

`-d` is detached mode. `--name` provides a convenient identifier. `-p 3000:3000` maps **host port 3000** to **container port 3000**. The `EXPOSE 3000` line in the Dockerfile **does not publish the port by itself**.

Stop and remove the container:

```
docker stop localloop
docker rm localloop
```

The image still exists. Starting a new container also starts a new in-memory listing collection. Persistent data requires a different design (database/storage).

6. Run with Docker Compose

`compose.yaml` describes a single `api` service. This intentionally prepares students for multi-service projects without introducing a database in the first Docker lecture.

```
docker compose config
docker compose up --build -d
docker compose ps
docker compose logs api
```

Test `http://localhost:3000/health`. Then clean up:

```
docker compose down
```

Do **not** use `docker compose down -v` casually in future stateful projects: the `-v` flag can delete named volumes and stored data.

7. Distribute an image through Docker Hub (optional)

Create or sign in to your Docker Hub account. Substitute your own username for `YOUR_ID`:

```
docker login
docker tag localloop-api:1.0 YOUR_ID/localloop-api:1.0
docker push YOUR_ID/localloop-api:1.0
```

A teammate with access can execute:

```
docker pull YOUR_ID/localloop-api:1.0
docker run --rm -p 3001:3000 YOUR_ID/localloop-api:1.0
```

Test `http://localhost:3001/health`. Do not push private code or image layers containing passwords/secrets. Registry access and whether a repository can be public or private depend on your account and institution's policies.

8. Offline alternative (when Docker Hub is blocked)

Sender:

```
docker save -o localloop-api-1.0.tar localloop-api:1.0
```

Receiver, after copying the tar file through an approved channel:

```
docker load -i localloop-api-1.0.tar
docker run --rm -p 3001:3000 localloop-api:1.0
```

9. Troubleshooting

Issue	What to try
`Cannot connect to the Docker daemon`	Start Docker Desktop/Engine; run `docker info`.
`port is already allocated`	Stop local Node or other container; alternatively `-p 3001:3000`.
`failed to read Dockerfile`	Confirm current directory is `localloop-api`.
`npm ci` fails	Ensure the supplied `package-lock.json` is present and matches `package.json`.

Issue	What to try
Browser cannot connect	Check `docker ps`, `docker logs localloop`, and host/container ports.
Data disappeared	Expected: demo storage is in-memory. Container replacement does not persist it.
`docker push` denied	Check login, image namespace, repository permission and network.

10. Extension tasks

1. **Beginner:** add `GET /api/listings/available` and a passing automated test; tag the rebuilt image `1.1`.
2. **Intermediate:** support a validated food category in the listing API.
3. **Intermediate:** add an explicit container healthcheck (research Docker `HEALTHCHECK` or Compose `healthcheck`).
4. **Advanced:** design persistent LocalLoop storage with a database service; test recovery after container recreation.

Suggested evidence: a Git commit, test output, Docker build command, working browser/API response, and an explanation of the new image tag.

11. Questions to answer in your own words

- How is a Dockerfile different from an image? How is an image different from a container?
- Why do we copy `package*.json` before the application source in the Dockerfile?
- What is the difference between `EXPOSE 3000` and `-p 3000:3000`?
- Which LocalLoop requirements are **not** addressed by this minimal teaching project?
- What would you change before deploying a real community food rescue service?

Sources

- COMP3028 Spring 2026 LocalLoop case study, supplied in the unit.
- Docker Node.js guide: https://docs.docker.com/guides/nodejs/
- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Docker Compose Quickstart: https://docs.docker.com/compose/gettingstarted/
- Docker image distribution: https://docs.docker.com/docker-hub/repos/manage/hub-images/push/
- Node.js release information: https://nodejs.org/en/about/previous-releases